
Syllabus for Data Structures and Algorithms

PGCP-ESD February 2026 Batch

Duration: 120 hours (42 theory hours + 48 lab hours + 30 hours self study)

Objective: Reinforcing knowledge of problem-solving techniques, data structure concepts, and analysis of different algorithms using C.

Prerequisites: Knowledge of programming in C

Evaluation: 100 Marks (Theory exam – 40%, Lab exam – 40%, Internals & Mini-project – 20%)

Textbook:

Data Structures and Algorithms in C, Mark Allen Weiss, Published by Pearson

References:

- Problem Solving: Best Strategies to Decision Making, Critical Thinking, and Positive Thinking by Thomas Richards / Kindle Edition
- Data Abstraction and Problem Solving with C: Walls and Mirrors by Janet Prichard, Frank M. Carrano / Pearson
- Introduction to Algorithms by Cormen, Leiserson, Rivest, and Stein

(Note: Each Session is of 2 hours)

Session 1: Introduction to Data Structures (2T + 2L +1SL)

Lecture:

- Introduction to Data Structures and Abstract Data Types (ADTs)
- Linear Data Structures: Arrays and Linked Lists
- Non-Linear Data Structures: Trees and Graphs

Lab:

- Array Operations Program
-

Sessions 2-3: Algorithms and Complexity Analysis (4T + 4L +2SL)

Lecture:

- Introduction to Algorithms and Algorithm Analysis
- Fundamental Abstract Data Types (ADTs)
- Complexity analysis of algorithms (Big O notation)

Lab:

- Analyze complexity of loops and recursive algorithms
-

Sessions 4-5: Object-Oriented Design and Basic Data Structures (4T + 6L +3SL)

Lecture:

- Object-Oriented design: Abstract Data Types (ADTs)
- Basic Data Structures: Arrays, Stacks, Queues, Circular Queues

Lab:

- Implement stack using arrays
 - Implement queues with different insertion methods
 - Implement circular queue
-

Sessions 6-7: Introduction to Linked Lists (4T + 4L +3SL)

Lecture:

- Introduction to Linked Lists
- Singly Linked Lists: Operations and Implementation

Lab: - Implement singly linked list operations

Sessions 8-9: Doubly Linked Lists and Advanced Linked Lists (4T + 6L +3SL)

Lecture:

- Introduction to Doubly Linked Lists
- Circular Linked Lists

Lab:

- Implement doubly linked list operations
 - Implement circular linked list operations
-

Sessions 10-11: Stacks and Queues (4T + 4L +3SL)**Lecture:**

- Stacks: Introduction and Implementation
- Queues: Introduction and Implementation

Lab:

- Implement stack using linked lists
 - Implement queue using linked lists
-

Sessions 12-13: Searching Algorithms (4T + 4L +3SL)**Lecture:**

- Objectives of Searching: Sequential Search, Binary Search

Lab:

- Implement sequential search
 - Implement binary search
-

Sessions 14-15: Hashing and Hash Tables (4T + 4L +3SL)**Lecture:**

- Hashing & Introduction to hash tables
- Hash Functions and Collision Resolution Techniques

Lab:

- Implement hashing techniques in C programs
 - Write a program to implement a hash table
-

Sessions 16-17: Sorting Algorithms I (4T + 6L +3SL)

Lecture:

- Introduction to sorting: Selection sort, Insertion sort, Bubble sort

Lab:

- Implement insertion sort
 - Implement selection sort
 - Implement bubble sort
-

Sessions 18-19: Sorting Algorithms II (4T + 4L +3SL)

Lecture:

- Advanced sorting: Heapsort, Mergesort, Quicksort
- Analysis of sorting algorithms

Lab:

- Implement heapsort
 - Implement mergesort
 - Implement quicksort
-

Sessions 20-21: Trees (4T + 4L +3SL)

Lecture:

- Binary Trees and Binary Search Trees (BST)
- Balanced Trees: AVL Trees and Red-Black Trees

Lab:

- Implement binary search tree operations
- Implement AVL tree operations
- Implement red-black tree operations

This revised syllabus includes additional sessions to cover more topics in detail, ensuring a thorough understanding of data structures and algorithms for the students.

Self Learning Topics

1. Advanced Problem-Solving Techniques

- Problem decomposition strategies
- Identifying patterns in problems
- Greedy thinking basics
- Divide-and-conquer mindset
- Backtracking intuition (without heavy math)

2. Recursion Mastery (Beyond Basics)

- Converting recursion to iteration
- Tail recursion concept
- Recursion vs stack memory trade-offs
- Common recursion pitfalls

3. Advanced Array Techniques

- Sliding window technique
- Two-pointer technique
- Prefix sum concept
- Frequency counting using arrays

4. String-Based Problem Solving

- String parsing techniques
- Pattern matching basics
- Palindrome checking strategies
- Anagram detection logic

5. Bit Manipulation for DSA

- Bit masking tricks
- Counting set bits
- Power-of-two checks
- XOR-based problem solving
- Bitwise tricks for optimization

6. Mathematical Foundations for DSA

- GCD & LCM algorithms
- Prime number checks
- Modular arithmetic basics
- Overflow-safe arithmetic

7. Hashing – Practical Enhancements

- Designing better hash functions
- Handling collisions efficiently
- Hashing vs sorting trade-offs
- Real-world hash table performance issues

8. Tree Problem-Solving Techniques

- Recursive tree thinking
- Height vs depth problems
- Tree property validation
- Tree-based problem patterns

9. Algorithmic Thinking & Optimization

- Brute force → optimized solution flow
- Identifying redundant computations
- Space-time trade-off reasoning
- Early termination techniques

10. Debugging & Dry-Run Skills

- Manual dry-running of algorithms
- Identifying logic errors
- Debugging infinite loops
- Off-by-one error detection

11. Complexity Estimation Skills

- Estimating runtime before coding
- Worst-case vs average-case reasoning
- Practical vs theoretical complexity
- Hidden costs in recursion and memory

12. Memory-Aware Data Structures

- Stack vs heap usage in DSA programs
- Memory overhead of different data structures
- Cache friendliness (basic awareness)

13. Competitive Programming Basics

- Input/output optimization
- Handling large inputs
- Constraints-based algorithm selection
- Writing efficient loops

14. Interview-Oriented DSA Skills

- Explaining logic clearly
- Trade-off discussion

- Writing clean, readable code
- Handling edge cases

15. Real-World Applications of DSA

- DSA usage in operating systems
- DSA usage in networking
- DSA usage in embedded systems
- Choosing practical over theoretical solutions